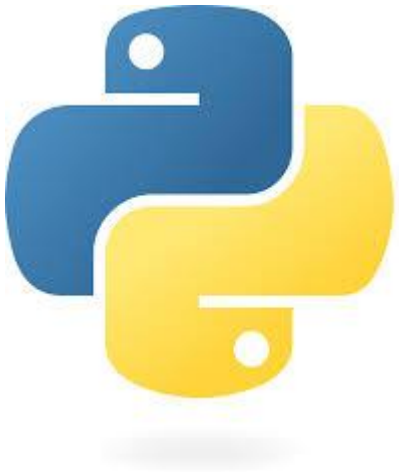
Python



PROGRAMMING FOR EVERYONE

Eng. Omar Shaqra

Pandas – Data Manipulation & Analysis in Python

What is Pandas?

- Open-source library built on top of **NumPy**
- Provides high-performance, easy-to-use data structures
- Designed for **tabular / labeled data** (like Excel, SQL tables, CSV files)

Main Objects

- **Series** → 1-dimensional labeled array (like a column)
- **DataFrame** → 2-dimensional labeled data structure (like a table / spreadsheet

Why use Pandas?

- Clean, powerful syntax
- Handles missing data very well
- Excellent for data cleaning, transformation, exploration
- Works beautifully with NumPy, Matplotlib, Scikit-learn, etc.
- Reads/writes almost any data format (csv, excel, json, sql, parquet, ...)

Pandas – Data Manipulation & Analysis in Python

Quick comparison – Python list vs Pandas Series vs DataFrame

```python
# Regular Python
names = ["Ahmed", "Sara", "Omar"]
ages  = [25, 32, 19]

# Pandas Series
import pandas as pd

s = pd.Series([25, 32, 19], index=["Ahmed", "Sara", "Omar"])
print(s)

# Pandas DataFrame (most common use)
df = pd.DataFrame({
    "name": ["Ahmed", "Sara", "Omar"],
    "age":  [25, 32, 19],
    "city": ["Cairo", "Alex", "Giza"]
})
print(df)
```

```
Ahmed    25
Sara     32
Omar     19
dtype: int64
    name  age   city
0  Ahmed   25  Cairo
1   Sara   32   Alex
2   Omar   19   Giza
```

Pandas gives names to rows and columns → much more expressive than plain NumPy arrays or lists

Pandas Series

What is a Series?

- A **one-dimensional** labeled array
- Can hold any data type (integers, floats, strings, objects, …)
- Has two main parts:
  - **Values** (the data)
  - **Index** (labels for the values — can be numbers, strings, dates, …)

```python
import pandas as pd

# 1. From a list (default integer index)
s1 = pd.Series([10, 25, 18, 42])
print(s1)
# 0    10
# 1    25
# 2    18
# 3    42
# dtype: int64

# 2. With custom index
s2 = pd.Series([85, 92, 78, 95],
               index=['Ahmed', 'Sara', 'Omar', 'Lina'])
print(s2)
# Ahmed    85
# Sara     92
# Omar     78
# Lina     95
# dtype: int64
```

Important Attributes & Methods

```python
print(s2.index)      # Index(['Ahmed', 'Sara', 'Omar', 'Lina'], dtype='object')
print(s2.values)     # array([85, 92, 78, 95])
print(s2.dtype)      # int64
print(s2.shape)      # (4,)
print(s2.size)       # 4
print(s2.name)       # None (you can set it: s2.name = 'Midterm Scores')
```

Pandas Series

Accessing Elements

```
# By position (like list / NumPy)
print(s2[0])          # 85
print(s2[-1])         # 95

# By label (preferred way)
print(s2['Sara'])     # 92
print(s2[['Omar', 'Lina']])   # returns a new Series

# Slicing (works with labels too!)
print(s2['Ahmed':'Omar'])     # includes both ends when using labels
```

Basic Operations (Vectorized – like NumPy)

```
print(s2 + 5)               # adds 5 to every value
print(s2 * 1.1)             # 10% bonus
print(s2 > 85)              # boolean Series
print(s2[s2 > 85])          # filter high scores
```

Pandas DataFrame

What is a DataFrame?

- A **2-dimensional**, **labeled** data structure
- Like a table, spreadsheet, or SQL table
- Consists of **rows** and **columns**
- Each **column** is a **Series**
- Can hold **different data types** in different columns

Key Features

- Labeled rows (**index**) and labeled columns (**columns**)
- Handles **missing data** very well (NaN)
- Supports powerful **indexing**, **slicing**, **filtering**, **grouping**, **joining**, etc.
- Built on top of NumPy → fast numerical operations

Pandas DataFrame

Creating a DataFrame

```python
import pandas as pd

# 1. From a dictionary (most common & recommended)
df1 = pd.DataFrame({
    'name':    ['Ahmed', 'Sara', 'Omar', 'Lina', 'Khaled'],
    'age':     [25, 32, 19, 28, 45],
    'city':    ['Cairo', 'Alex', 'Giza', 'Cairo', 'Mansoura'],
    'score':   [88.5, 92.0, 78.0, 85.5, 67.0]
})

print(df1)
```

```
name  age      city  score
0   Ahmed   25     Cairo   88.5
1    Sara   32      Alex   92.0
2    Omar   19      Giza   78.0
3    Lina   28     Cairo   85.5
4  Khaled   45  Mansoura   67.0
```

From a list of dictionaries

```python
data = [
    {'name': 'Ali', 'age': 22, 'score': 79},
    {'name': 'Nour', 'age': 31, 'score': 91},
    {'name': 'Yara', 'age': 27, 'score': 84}
]
df2 = pd.DataFrame(data)
```

```
   name  age  score
0   Ali   22     79
1  Nour   31     91
2  Yara   27     84
```

Pandas DataFrame

From NumPy array + column names

```
import numpy as np
arr = np.random.randint(50, 100, (5, 3))
df3 = pd.DataFrame(arr, columns=['Math', 'Physics', 'Chemistry'])
```

```
arr [[93 82 62]
 [97 62 68]
 [71 64 74]
 [64 52 73]
 [68 66 74]]
df3    Math  Physics  Chemistry
0     93       82         62
1     97       62         68
2     71       64         74
3     64       52         73
4     68       66         74
```

Empty DataFrame (you can add columns later)

```
df_empty = pd.DataFrame()
df_empty['id'] = [1, 2, 3]
df_empty['value'] = [10.5, 20.1, 15.3]
```

```
   id  value
0   1   10.5
1   2   20.1
2   3   15.3
```

Pandas DataFrame

Essential DataFrame Attributes

& Methods

```
print(df1.shape)        # (5, 4) → rows, columns
print(df1.index)        # RangeIndex(start=0, stop=5, step=1)
print(df1.columns)      # Index(['name', 'age', 'city', 'score'], dtype='object')
print(df1.dtypes)       # data type of each column
print(df1.info())       # very useful summary
print(df1.head(3))      # first 3 rows (default 5)
print(df1.tail(2))      # last 2 rows
```

Quick Access Examples

```
# Select one column → returns Series
print(df1['name'])

# Select multiple columns
print(df1[['name', 'age']])

# Select row by position
print(df1.iloc[0])       # first row

# Select row by label (if index is meaningful)
# (we'll cover .loc vs .iloc properly in next slides)
```

Reading Data with Pandas

Almost every data analysis starts with loading data from files: CSV, Excel, JSON, SQL databases, clipboard, etc.

Pandas makes it extremely easy and powerful.

```
import pandas as pd


df = pd.read_csv('students.csv')

print("the first 5 lines : ")
print(df.head())

print("\ninformation")
print(df.info())

print("\nstatistics")
print(df.describe())

print("\nthe nums of missing values in each column:")
print(df.isna().sum())
```

Quick Data Exploration

Why explore first?

- Understand structure & size
- Check data types
- Spot missing values early
- See basic statistics
- Discover problems (wrong types, duplicates, outliers)

```
print(df.shape)          # (rows, columns) e.g. (8, 8)
print(df.columns)        # list of column names
print(df.index)          # row labels (usually RangeIndex)

df.head(5)               # first 5 rows (default)
df.tail(3)               # last 3 rows
df.sample(4)             # random 4 rows (good for variety)


df.info()                # most important!
# Shows: column names, non-null count, dtype, memory usage

df.dtypes                # data types of each column

df.describe()            # statistics for numeric columns only
df.describe(include='object')   # for categorical/string columns
```

Quick Data Exploration

Missing values & duplicates

```
df.isna().sum()            # number of missing values per column
df.isna().mean() * 100     # percentage of missing values

df.duplicated().sum()      # number of duplicate rows
df.duplicated(subset=['Name', 'Age']).sum()   # check specific columns
```

Value counts & unique values

```
df['City'].value_counts()           # frequency of each city
df['Subject'].value_counts(normalize=True)   # proportions

df['Grade'].nunique()                # number of unique values
df['Grade'].unique()                 # list of unique values (may be long)
```

Quick numeric insights

```
df['Grade'].min()
df['Grade'].max()
df['Grade'].mean()
df['Grade'].median()
df['Grade'].std()
```

Selecting & Accessing Data

Selecting Columns

```
# Single column → returns a Series
df['Name']
df.Name                # dot notation (only if column name is valid identifier)

# Multiple columns → returns a DataFrame
df[['Name', 'Grade', 'City']]

# All columns except one or two
df.drop(columns=['Student ID', 'Score Date'])
# or
df.loc[:, ~df.columns.isin(['Student ID', 'Score Date'])]
```

Selecting & Accessing Data

Selecting Rows & Columns – loc vs iloc

```
# .loc  → label-based (uses index labels and column names)
df.loc[0]                    # row with index label 0
df.loc[2:5]                  # rows 2 to 5 (inclusive!)
df.loc[:, 'Name':'City']     # all rows, columns from Name to City (inclusive)
df.loc[df['Grade'] > 80, ['Name', 'Grade']]   # rows where Grade > 80 + specific columns

# .iloc → position-based (integer position, like NumPy)
df.iloc[0]                   # first row
df.iloc[2:5]                 # rows 2,3,4 (stop is exclusive)
df.iloc[:, 1:4]              # columns 1,2,3 (Name, Age, Grade)
df.iloc[-3:]                 # last 3 rows
```

Selecting & Accessing Data

Fast access to single value

```
# Recommended for single value access (faster & cleaner)
df.at[3, 'Name']            # label-based
df.iat[3, 1]                # position-based (row 3, column 1)

# Avoid this in loops (slow)
df.loc[3]['Name']
```

Setting values

```
# Change single value
df.at[3, 'Grade'] = 82

# Change multiple values with condition
df.loc[df['City'] == 'Cairo', 'City'] = 'Cairo (Capital)'

# Add new column
df['Status'] = df['Grade'].apply(lambda x: 'Pass' if x >= 70 else 'Fail')
```

Filtering & Boolean Indexing

Basic filtering

```
# Single condition
high_grade = df['Grade'] >= 80
df[high_grade]                  # same as:
df[df['Grade'] >= 80]

# Show only names and grades of students with high grades
df[df['Grade'] >= 80][['Name', 'Grade']]
```

Multiple conditions

```
# AND → &
df[(df['Grade'] >= 80) & (df['City'] == 'Cairo')]

# OR → |
df[(df['Grade'] >= 90) | (df['Passed'] == True)]

# NOT → ~
df[~(df['City'] == 'Cairo')]          # everyone NOT from Cairo

# Combine all
df[(df['Grade'] > 75) & (df['Passed'] == True) & (df['City'].isin(['Cairo', 'Alexandria']))]
```

Filtering & Boolean Indexing

useful filtering methods

```
# isin() - multiple exact matches
df[df['Subject'].isin(['Math', 'Physics'])]

# str methods (great for text columns)
df[df['Name'].str.startswith('A')]           # names starting with A
df[df['Name'].str.contains('ar', case=False)] # contains "ar" anywhere (case insensitive)

# between() - range check
df[df['Grade'].between(70, 89)]             # grades from 70 to 89 inclusive

# isnull() / notnull()
df[df['Grade'].isnull()]                    # students with missing grade
df[df['Score Date'].notnull()]
```

Filtering + selecting columns together

```
# Best practice: use loc to avoid SettingWithCopyWarning
df.loc[df['Grade'] >= 85, ['Name', 'Grade', 'Subject']]

# Sort the result
df.loc[df['Grade'] >= 85, ['Name', 'Grade']].sort_values('Grade', ascending=False)
```

Filtering & Boolean Indexing

Query method

```
# Very clean for complex conditions
df.query("Grade >= 80 and City == 'Cairo'")
df.query("Grade > @min_grade and Passed == True", local_vars={'min_grade': 75})
```

Important Warning – SettingWithCopyWarning

```
# BAD - may not change the original DataFrame
df[df['Grade'] < 70]['Grade'] = 70

# Correct ways:
df.loc[df['Grade'] < 70, 'Grade'] = 70

# or
low = df['Grade'] < 70
df.loc[low, 'Grade'] = 70
```

Handling Missing Data

What are missing values in Pandas?

- Represented as NaN (Not a Number) or None
- Common symbols: NaN, null, empty string, NA, -, ?, etc.
- Missing values can break calculations, plots, models

```
df.isna()                # returns boolean DataFrame (True where missing)
df.notna()               # opposite

df.isna().sum()          # count of missing values per column
df.isna().sum().sum()    # total missing values in the whole DataFrame

df.isna().mean() * 100   # percentage of missing values per column
```

Handling Missing Data

Dropping Missing Values

```
# Drop any row with at least one missing value
df.dropna()

# Drop rows only if all values are missing
df.dropna(how='all')

# Drop columns that have missing values
df.dropna(axis=1)

# Drop rows only if certain columns have missing values
df.dropna(subset=['Grade', 'Score Date'])
```

Interpolation (smooth filling)

```
# Linear interpolation (great for numeric sequences)
df['Grade'].interpolate()

# Or on whole DataFrame for numeric columns
df.interpolate()
```

Filling Missing Values

```
# Fill with a constant value
df['Grade'].fillna(0)
df.fillna(0)                # fill entire DataFrame

# Fill with mean / median (common for numeric columns)
df['Grade'].fillna(df['Grade'].mean())
df['Grade'].fillna(df['Grade'].median())

# Forward fill / backward fill (good for time series)
df['Score Date'].ffill()    # use previous valid value
df['Score Date'].bfill()    # use next valid value

# Fill with value from another column
df['Age'].fillna(df['Age'].mean())

# Fill different columns with different values
df.fillna({'Grade': 60, 'City': 'Unknown'})
```

Handling Missing Data

Replacing non-standard missing values

```
# Replace weird placeholders with NaN during loading or after
df.replace(['N/A', '-', '?', 'missing'], pd.NA, inplace=True)
```

Best Practice Patterns

```
# Typical cleaning sequence
df = df.replace(['', 'NA', 'N/A', '-'], pd.NA)     # standardize missing
df = df.dropna(subset=['Name', 'Student ID'])      # critical columns
df['Grade'] = df['Grade'].fillna(df['Grade'].mean()) # numeric imputation
df['City'] = df['City'].fillna('Unknown')          # categorical imputation
```

Adding & Removing Columns & Rows

Simple ways to add columns

```
# Constant value
df['School'] = 'Cairo University'

# From existing columns (vectorized - fastest)
df['Age_in_5_years'] = df['Age'] + 5

# Using calculation
df['Grade_Percent'] = df['Grade'] / 100

# Conditional column (very common)
df['Status'] = df['Grade'].apply(lambda x: 'Pass' if x >= 70 else 'Fail')

# Using np.where (faster than apply for simple cases)
import numpy as np
df['Excellent'] = np.where(df['Grade'] >= 90, 'Yes', 'No')
```

Adding & Removing Columns & Rows

Adding Columns with More Complex Logic

```
df['Grade_Letter'] = pd.cut(
    df['Grade'],
    bins=[0, 70, 80, 90, 100],
    labels=['F', 'C', 'B', 'A'],
    include_lowest=True
)
```

```
def map_grade(x):
    if 90 <= x <= 100:
        return 'A'
    elif 80 <= x < 90:
        return 'B'
    elif 70 <= x < 80:
        return 'C'
    else:
        return 'F'

df['Grade_Letter'] = df['Grade'].apply(map_grade)
```

Removing Columns

```
# Drop one column
df = df.drop(columns='School')

# Drop multiple columns
df.drop(columns=['Age_in_5_years', 'Grade_Percent'], inplace=True)

# Drop columns by condition (e.g. almost empty)
df = df.loc[:, df.notna().mean() > 0.5]   # keep columns with >50% non-missing
```

Adding & Removing Columns & Rows

Removing Rows

```
# Drop rows with any missing values
df.dropna()

# Drop rows based on condition
df = df[df['Grade'] >= 50]                    # keep only passing grades
df = df[~df['City'].isin(['Unknown', 'N/A'])]  # exclude bad cities

# Drop duplicate rows
df = df.drop_duplicates(subset=['Name', 'Age'], keep='first')
```

Adding Rows

```
# Add single row
new_row = {'Student ID': 9, 'Name': 'Ziad', 'Age': 24, 'Grade': 87, 'Subject': 'Math',
           'City': 'Cairo', 'Passed': True, 'Score Date': pd.Timestamp('2025-04-01')}
df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)
```

Apply, Map & Vectorized Operations

Three main ways to transform columns

- **Vectorized operations** (fastest & recommended)
- **map()** (good for Series / simple replacements)
- **apply()** (flexible but slower – use when needed)

Vectorized Operations

- Operations applied directly to entire columns (no loops, uses NumPy under the hood)

```
# Examples
df['Grade'] = df['Grade'] + 5                    # add bonus to all
df['Age_Next_Year'] = df['Age'] + 1

df['Grade_Percent'] = df['Grade'] / 100
df['Final_Score'] = df['Grade'] * 0.6 + 40       # weighted + base
```

Apply, Map & Vectorized Operations

```python
# Conditional vectorized with np.where
import numpy as np
df['Result'] = np.where(df['Grade'] >= 70, 'Pass', 'Fail')

# Multiple conditions with np.select
conditions = [
    df['Grade'] >= 90,
    df['Grade'] >= 80,
    df['Grade'] >= 70
]
choices = ['A', 'B', 'C']
df['Letter'] = np.select(conditions, choices, default='D')
```

map() – For Series / Simple Mapping

```python
# Replace values using dictionary
city_map = {'Cairo': 'Cairo Capital', 'Alexandria': 'Alex', 'Giza': 'Giza'}
df['City_Short'] = df['City'].map(city_map)

# Or function
df['Name_Upper'] = df['Name'].map(str.upper)

# With get() - safe for missing keys
df['City_Type'] = df['City'].map({'Cairo': 'Big', 'Alexandria': 'Big'}).fillna('Small')
```

Apply, Map & Vectorized Operations

apply() – Most Flexible (but slower)

```python
# Apply function to each value in a column
def grade_category(g):
    if g >= 90: return 'Excellent'
    elif g >= 75: return 'Good'
    elif g >= 60: return 'Average'
    else: return 'Needs Work'

df['Category'] = df['Grade'].apply(grade_category)

# Apply to entire row (axis=1)
def student_summary(row):
    return f"{row['Name']} - {row['Grade']} ({row['Subject']})"

df['Summary'] = df.apply(student_summary, axis=1)
```

Method	Speed	Use case	Best for
Vectorized	Fastest	Simple math, np.where, pd.cut	Most transformations
map()	Very fast	Dictionary mapping, simple functions	Series only
apply()	Slowest	Complex logic, row-wise operations	When nothing else works

Sorting & Ranking

Sorting Rows with sort_values()

```python
# Sort by one column (ascending by default)
df.sort_values('Grade')

# Descending order
df.sort_values('Grade', ascending=False)

# Sort by multiple columns
df.sort_values(['City', 'Grade'], ascending=[True, False])
# → first by City A→Z, then inside each city highest Grade first
```

```python
# Keep the sorted order (inplace or assign)
df = df.sort_values('Grade', ascending=False)

# Show top 5 students
df.sort_values('Grade', ascending=False).head(5)

# Sort by index (useful after filtering/grouping)
df.sort_index()

# Sort by column names (alphabetically)
df.sort_index(axis=1)
```

Sorting & Ranking

Ranking with rank()

```
# Add a rank column (1 = highest)
df['Rank'] = df['Grade'].rank(ascending=False)
```

```
# Handle ties (different methods)
df['Rank_dense']   = df['Grade'].rank(ascending=False, method='dense')    # 1,2,2,3
df['Rank_min']     = df['Grade'].rank(ascending=False, method='min')      # 1,2,2,4
df['Rank_max']     = df['Grade'].rank(ascending=False, method='max')      # 1,3,3,4
df['Rank_average'] = df['Grade'].rank(ascending=False, method='average')  # 1,2.5,2.5,4
```

Getting top/bottom N rows

```
# Top 3 highest grades
df.nlargest(3, 'Grade')

# Bottom 3 lowest grades
df.nsmallest(3, 'Grade')
```

Lab

- Load the file students.csv and make sure Score Date is parsed as datetime.
- Print the shape, columns list, and data types of the DataFrame.
- Show the first 7 rows and the last 3 rows.
- Show how many missing values in each column.
- What is the average, minimum, and maximum Grade?
- How many unique cities are there? List them.
- Use value_counts() to show how many students per Subject.

- Select only the columns: Name, Grade, Subject, City.
- Show all students who Passed and have Grade ≥ 85.
- Show students whose Name contains the letter "a" or "A" (case insensitive).
- Show students in Cairo or Alexandria with Grade < 80.
- Show students with Grade between 80 and 90 (inclusive).
- Using .loc, change Khaled's grade from 55 to 68.
- Add a new column Is_Excellent that is True if Grade ≥ 90, False otherwise (vectorized).

Lab

- Fill missing Grade values with the average Grade.
- Fill missing Score Date values with the most common date (mode).
- Add a new column Final_Grade = Grade + 5 (bonus points).
- Create a column Grade_Category using pd.cut with bins:
- 0–69 → "Fail"
- 70–79 → "Average"
- 80–89 → "Good"
- 90–100 → "Excellent"
- Remove the column Student ID.
- Keep only students who Passed.
- Sort the DataFrame by Grade descending.
- Sort by City ascending, then inside each city sort by Grade descending.
- Add a column Rank that ranks students by Grade (1 = highest, handle ties with 'min').
- Show the top 4 students with the highest grades (use nlargest).
- Show the 3 youngest students (use nsmallest on Age).
- Find the student(s) with the highest Grade and show their Name and Grade.

Lab

- After filling missing grades, recalculate the average Grade per Subject.
- Create a new column City_Avg that shows the average grade of each student's city (using transform).
- Show the subject with the highest average grade.